

ESPECIALIZACIÓN EN DISEÑO Y DESARROLLO DE VIDEOJUEGOS

ACA 2

Presentado por:

Brayan Steven Castelblanco Utria

Jhoan Andres Castelblanco Utria

Luis Enrique Castelblanco Utria

Docente:

Mateo Mancera Camacho

Asignatura:

Diseño y construcción de ambientes interactivos en motores de videojuegos 54ES1

Bogotá D.C, Septiembre de 2026

1. Introducción

El rendimiento en tiempo real es un factor crítico en el desarrollo de videojuegos. Un juego puede tener buenos gráficos y jugabilidad, pero si no es fluido, la experiencia del jugador se ve afectada. Este trabajo documenta el proceso de perfilado, medición y optimización realizado sobre un prototipo desarrollado en Unity, con el objetivo de eliminar los principales cuellos de botella y garantizar una experiencia estable.

El proyecto parte de una premisa clave: los problemas de rendimiento no suelen estar en "scripts lentos", sino en la asignación excesiva de memoria y el churn de objetos, que generan pausas del recolector de basura y tirones perceptibles para el jugador. Para demostrarlo, se implementó un medidor de rendimiento propio y se diseñó un escenario de medición idéntico antes y después de las optimizaciones.

El documento se estructura en tres bloques: primero, la metodología de medición (herramienta, escenario y micro-benchmarks); segundo, los tres ajustes aplicados (raycast sin asignaciones, voces de audio reutilizables y cacheo de referencias), cada uno con evidencia medida; y tercero, los elementos del juego (IA del arquero, interfaz uGUI y Blend Tree de animación), mostrando cómo se integran y qué decisiones técnicas se tomaron.

En síntesis, este trabajo demuestra que la optimización en videojuegos se mide, no se estima, y que las ganancias más significativas vienen de eliminar la basura generada por cuadro.

2. Metodología de medición

2.1 Herramienta usada

Se implementó un medidor propio, Assets/Scripts/MedidorRendimiento.cs, que mide durante una ventana de tiempo y entrega:

- Tiempo de cuadro promedio (ms) y FPS;
- Draw calls, SetPass y triángulos (estadísticas de render del editor);
- Objetos en escena al inicio y al final de la ventana (mide el churn de objetos, que es exactamente lo que terminaba limpiando el recolector de basura).

Una métrica se descartó a propósito: GC.GetTotalAllocatedBytes no existe en el perfil de .NET de Unity, y ProfilerRecorder devolvía 0 en el editor, así que las asignaciones se midieron con benchmarks aislados (sección 2.3), que dan el número por patrón sin ruido del resto del juego.

2.2 Escenario de medición (idéntico antes y después)

- Se inicia la ronda (la cancha se cierra, aparece la pelota, el arquero entra en su IA, el HUD y la cámara quedan trabajando).
- Se lanza una ráfaga de 200 sonidos 3D (el patrón que usaba el balón al rebotar).
- Se mide una ventana de 6 s (con 0,5 s de calentamiento descartados).

Se usó el mismo escenario en las dos corridas para que la comparación sea válida.

Nota honesta sobre el tiempo de cuadro: en el Editor el cuadro está dominado por el render del editor (≈ 16 ms $\approx$ los 60 Hz de vsync), así que el tiempo de cuadro apenas se mueve. Por eso la evidencia fuerte está en las asignaciones y en el churn de objetos, que es lo que el perfilado detectó y lo que las optimizaciones atacan. En una build final esa basura se convierte en pausas del recolector, que es justamente lo que se evita.

2.3 Micro-benchmarks (medición aislada de cada patrón)

Para medir una sola cosa por vez se corrieron bucles cerrados comparando el patrón viejo contra el nuevo, midiendo la diferencia del heap de mono (con una recolección forzada antes de cada tramo) y la cantidad de objetos creados. Así se obtiene el costo por llamada, sin depender del resto del juego.

3. Evidencia de profiling: antes y después

3.1 Medición en juego (misma escena, mismo escenario, 6 s)

El tiempo de cuadro casi no cambia porque en el editor está dominado por el render/vsync (sección 2.2), pero el churn de objetos —que es lo que genera pausas del recolector— bajó de 200 a 0 en la misma carga de trabajo.

Métrica	ANTES (sin optimizar)	DESPUÉS (optimizado)	Variación
Cuadros medidos	396	402	—
Tiempo de cuadro promedio	16,42 ms (66 FPS)	16,18 ms (67 FPS)	−0,24 ms
Draw calls	1264	1265	±0
SetPass calls	44	44	±0
Triángulos	75 883	75 883	±0
Objetos temporales creados/destruidos en la ventana	−203	−2	−99 %
Objetos que creó la ráfaga de 200 sonidos	200	0	−100 %

3.2 Micro-benchmarks por patrón (lo que justifica cada ajuste)

Benchmark	Patrón viejo	Patrón nuevo
Physics.RaycastAll vs Physics.RaycastNonAlloc (30 000 llamadas)	2 936 KB ($\approx$ 100 bytes por llamada)	12 KB ($\approx$ 0 bytes por llamada)
Cada llamada devuelve un array nuevo	Sí (verificado con ReferenceEquals)	No (buffer propio)
PlayClipAtPoint (200 sonidos)	200 objetos (GameObject + AudioSource por sonido)	0 objetos
GetComponent<T>() (300 000 llamadas)	31,2 ms ($\approx$ 0,10 μ s por llamada)	1 búsqueda por objeto

3.3 Capturas del Profiler

Las mediciones de arriba salen del medidor propio y de las estadísticas del editor. Para adjuntar la captura del Profiler con la misma comparación, el procedimiento es:

- Unity ► Window ► Analysis ► Profiler.
- Play, iniciar la ronda y lanzar la ráfaga de sonidos (sección 2.2).

En el Profiler: Record durante los 6 s, y mirar los módulos CPU (tiempo de cuadro) y Memory (GC Alloc / Managed Heap).

- Detener y capturar la pantalla del Profiler (Profiler_antes.png / Profiler_despues.png).

En el repositorio quedan además dos capturas reales del juego corriendo, que se usan en la sección 8 (interfaz construida): Assets/Screenshots/entregable_pregunta.png y Assets/Screenshots/entregable_ronda.png.

4. Ajuste 1 · Cámara: raycast sin asignaciones

Dónde: Assets/Scripts/CamaraTerceraPersona.cs (método LateUpdate).

Qué hacía antes: cada cuadro la cámara tiraba un rayo contra los obstáculos para no atravesar la malla, las tribunas ni los edificios, usando Physics.RaycastAll, que devuelve un array nuevo en cada llamada.

```
// ANTES: un array nuevo por cuadro (basura para el recolector)
RaycastHit[] impactos = Physics.RaycastAll(pivote, direccion.normalized, largo,
capasObstaculo, QueryTriggerInteraction.Ignore);
for (int i = 0; i < impactos.Length; i++) { ... }

// DESPUÉS: un buffer propio que se reutiliza todos los cuadros
private readonly RaycastHit[] impactosBuffer = new RaycastHit[16];

int encontrados = Physics.RaycastNonAlloc(pivote, direccion.normalized, impactosBuffer,
largo, capasObstaculo, QueryTriggerInteraction.Ignore);
for (int i = 0; i < encontrados; i++) { ... }
```

Justificación (medida): 100 bytes por llamada $\times$ 60 cuadros por segundo $\approx$ 6 KB por segundo de basura solo por esta función, y crece con la cantidad de colliders que toca el rayo. Con el buffer fijo el costo de asignación es 0: se reescribe el mismo array. Es el patrón que Unity recomienda para consultas físicas por cuadro.

5. Ajuste 2 · Audio: voces 3D reutilizables en vez de crear un objeto por sonido

Dónde: Assets/Scripts/GestorAudio.cs (sonidos 3D: bote del balón, portón de la reja).

Qué hacía antes: para que un sonido se escuchara desde el lugar del golpe se usaba `AudioSource.PlayClipAtPoint`, que crea un `GameObject` nuevo con su `AudioSource`, lo posiciona, lo reproduce y lo destruye al terminar el clip. En una ronda con el balón rebotando y chocando contra el arco y el cerco, eso son decenas de objetos por minuto.

// ANTES: un GameObject + AudioSource por cada sonido (y su Destroy después)

AudioSource.PlayClipAtPoint(clip, posicion, v);

// DESPUÉS: anillo de fuentes 3D creadas una sola vez y reutilizadas

AudioSource voz = fuentes3D[siguiente3D];

siguiente3D = (siguiente3D + 1) % fuentes3D.Length;

voz.transform.position = posicion;

voz.pitch = Random.Range(0.96f, 1.04f);

voz.PlayOneShot(clip, v);

Las voces se crean en `Awake()` (6 por defecto, configurable en el Inspector como `voces3D`) con `spatialBlend = 1` y atenuación lineal entre 2 y 25 m, así se conserva el audio posicional (mientras más lejos del golpe, más suave) sin crear ni destruir nada.

Justificación (medida): una ráfaga de 200 sonidos creaba +200 objetos; con el anillo de voces crea +0. Son 200 `GameObjects`, 200 `AudioSources` y sus `Destroy` que ya no existen: eso es menos trabajo de creación, de física/escena y sobre todo menos pausas del recolector, que son las que se sienten como tirones.

6. Ajuste 3 · Referencias cacheadas en lugar de GetComponent repetidos

Dónde: ConductorPelota.cs (el Rigidbody de la pelota) y Arquero.cs (el Animator).

Qué hacía antes:

ConductorPelota.FixedUpdate() llamaba a `pelota.GetComponent<Rigidbody>()` en cada paso de la física (50 veces por segundo) solo para saber si la pelota existía y estaba en juego.

Arquero buscaba su Animator con GetComponent cada vez que se lanzaba.

// ANTES (en FixedUpdate, 50 veces por segundo)

Rigidbody rb = pelota.GetComponent<Rigidbody>();

// DESPUÉS: una sola búsqueda por pelota, y solo si cambia la referencia

```
private Rigidbody RigidbodyPelota()
{
    if (pelota == null) return null;
    if (rbPelotaCache == null || pelotaDelCache != pelota)
    {
        rbPelotaCache = pelota.GetComponent<Rigidbody>();
        pelotaDelCache = pelota;
    }
    return rbPelotaCache;
}
```

Justificación (medida): una búsqueda cuesta $\approx 0,10 \mu s$, por lo que el ahorro por sí solo es chico (del orden de microsegundos por segundo). Se aplicó igual por dos motivos: (a) es la práctica correcta —nunca conviene buscar componentes dentro de un bucle que

corre todos los cuadros o todos los pasos de física—, y (b) el costo crecía en proporción al trabajo de la pelota, que es el objeto con más eventos del juego. La regla que quedó en el código: buscar una vez y guardar.

7. Comportamiento de IA implementado (el arquero)

El arquero (Assets/Scripts/Arquero.cs) es una IA reactiva con predicción: no está quieto esperando, y tampoco sabe de antemano a dónde va el tiro. Tiene tres comportamientos que se alternan según lo que detecta, y se mueve con un Rigidbody cinemático + MovePosition, así que empuja la pelota de verdad cuando la toca.

Comportamiento	Cuándo se activa	Qué hace
1. Acomodo (Update)	Estado normal, la pelota no viene	Se mueve constantemente por su área: un balanceo lateral de $\pm 0,30$ m a 0,6 Hz, siguiendo la posición de la pelota (85 % del costado por el que atacan). Si el balón se acerca, sale a achicar hasta 4,5 m.
2. Lectura y lanzamiento	La pelota supera la velocidad de disparo	Predice por dónde va a cruzar su línea, espera su tiempo de reacción y recién ahí se lanza hacia ese punto.
3. Salida a interceptar	La pelota rueda lenta hacia su arco	Sale a buscarla (4,2 m/s) en lugar de esperarla.

Parámetros reales del Inspector:

Parámetro	Valor	Para qué
mediaCancha	1,4 m	Cuánto se mueve a los costados (el arco mide 3,60 m:

		1,80 de media)
balanceoAmplitud / balanceoFrecuencia	0,30 m / 0,6 Hz	El balanceo constante del acomodo
seguimientoBalon	0,85	Cuánto sigue la posición de la pelota
distanciaCortaAngulo	20 m	Desde cuántos metros sale a achicar el ángulo
avanceMaximo	4,5 m	Cuánto puede adelantarse del arco
velocidadAcomodo / velocidadReaccion / velocidadSalida	4 / 5 / 4,2 m/s	Velocidades de cada comportamiento
retardo	0,22 s	Tiempo de reacción antes de lanzarse
errorMaximo	0,40 m	Error de cálculo del punto de cruce
toques	contador	Se suma cada vez que la pelota lo toca (lo usa el juez)

La predicción y su error (PredecirCruce):

$$zPredicho = pos.z + vel.z * t + Random.Range(-errorMaximo, errorMaximo);$$

Es decir: proyecta el cruce por la línea del arco y le suma un error aleatorio de $\pm 0,40$ m. Junto con el retardo de 0,22 s, esto hace que el arquero sea bueno pero no perfecto: se nota que lee el tiro y se lanza, y por eso los goles entran por los rincones o con tiros bien cargados.

Cómo se conecta con el juez de tiro: el arquero lleva el contador toques (se incrementa en OnCollisionEnter cuando lo golpeado es la pelota). Al resolver cada tiro, JuezTiro compara `arquero.toques > toquesAlTirar`: si cambió, el arquero la tocó y el resultado es ATAJADA; si no y tampoco fue gol, es FALLADO. Así el resultado sale de la simulación, no de un número al azar.

Audiovisual del arquero: cuando se lanza, el Animator dispara el trigger Izquierda o Derecha según el lado al que va y el clip inclina el cuerpo y la cabeza unos 30° (sección 9).

8. Interfaz construida (uGUI)

Toda la interfaz está armada con uGUI sobre un canvas UI_Juego y se conecta a los scripts por el Inspector. La guía de comandos cambia según el contexto, así que el jugador nunca ve teclas que no sirven.

Elemento	Ubicación	Qué muestra / hace
Guía de comandos (GuiaComandos)	Abajo a la derecha, siempre visible	Lista de teclas. En modo ronda oculta “Acción / Entrar” y agrega “Disparar (mantener para cargar)”, “R · Reponer la pelota” y “E · Salir de la ronda”.
Panel de pregunta (AvisoEntradaCancha)	Centro-abajo	“¿Quieres iniciar una ronda de tiros de castigo?” con las opciones E · Sí / Q · Ahora no, y la variante para salir. Entra con una animación suave y un clic.
Marcador de la ronda (IndicadorRonda)	Arriba a la derecha	5 círculos de tiros + 5 de atajadas. Gris = sin resolver, verde = acierto, rojo = fallo. Cada círculo que se pinta hace un pulso de escala y suena.
Barra de fuerza (BarraFuerza)	Abajo, centro	Aparece mientras se mantiene el clic y se llena de verde a rojo según la carga; al llegar al 100 % suena un aviso.
Mensaje de resultado (JuezTiro)	Centro de la pantalla	¡GOL! / ¡ATAJADA! / ¡FALLADO! con su color, su pop de escala y su sonido. Al terminar los 5 tiros muestra el resumen con F · Repetir la ronda y E · Salir.

Decisiones de diseño de la interfaz

- Una sola pregunta a la vez: el aviso de entrada y el de salida usan el mismo panel, así el jugador siempre responde con las mismas dos teclas (E/Q).
- Información redundante a propósito: el resultado se comunica tres veces (mensaje grande + color del círculo del marcador + sonido), para que se entienda sin mirar una zona puntual.
- El panel de la barra no lleva el script: BarraFuerza vive en un objeto siempre activo porque un objeto inactivo no ejecuta Update (detalle que ya está documentado en el propio script).
- Capturas reales del juego corriendo: Assets/Screenshots/entregable_pregunta.png (la pregunta E/Q en pantalla) y Assets/Screenshots/entregable_ronda.png (la ronda con el marcador de círculos pintado).

9. Blend Tree de animación usado (locomoción del jugador)

Un Blend Tree sirve para mezclar varios clips de forma continua según un parámetro, en vez de saltar de un estado a otro. Se usó para la locomoción del jugador: así el pase de quieto → caminando → corriendo es un degradé y no un cambio brusco.

Dónde: Assets/Animations/JugadorMovimiento.controller

Elemento	Valor
Estado	1 solo: Locomocion (es el estado por defecto)
Tipo de Blend Tree	Simple 1D (un solo parámetro de mezcla)
Parámetro	Desplazamiento (tipo float, 0 → 1)
Clips (hijos)	3, en los umbrales 0,00 / 0,45 / 1,00

writeDefaultValues	desactivado (para que no escriba ceros donde no debe)
--------------------	---

Clip	Umbral	Duración	En bucle	Qué anima (hijo Cuerpo)
Jugador_Quieto	0	0,80 s	Sí	Sin movimiento (pose base)
Jugador_Caminar	0,45	0,75 s	Sí	Balanceo vertical de 0,045 m, inclinación 3°, cabeceo lateral ±2,5°
Jugador_Correr	1	0,45 s	Sí	Balanceo vertical de 0,100 m, inclinación 8°, cabeceo lateral ±4,5°

Verificación en vivo de la mezcla (leyendo el peso real de cada clip con Animator.GetCurrentAnimatorClipInfo mientras se cambiaba el parámetro):

Desplazamiento	Resultado medido
0	Quieto 100 %
0,23	Quieto 50 % + Caminar 50 %
0,45	Caminar 100 %
0,73	Caminar 50 % + Correr 50 %
1	Correr 100 %

Es decir: el Blend Tree interpola de verdad entre los tres clips, y los umbrales están puestos para que el punto medio entre dos animaciones caiga a mitad de camino entre sus dos velocidades.

Cómo se alimenta el parámetro (JugadorTerceraPersona.ActualizarAnimacion):

```
float objetivo = Mathf.Clamp01(RapidezActual / Mathf.Max(0.01f, velocidadCorrer)); //  
0..1 real  
desplazamiento = Mathf.MoveTowards(desplazamiento, objetivo, 4f * Time.deltaTime); //  
suavizado  
if (Mathf.Abs(desplazamiento - desplazamientoEscrito) < 0.01f) return; // no repetir  
desplazamientoEscrito = desplazamiento;  
animador.SetFloat(HashDesplazamiento, desplazamiento); // hash, no string
```

Tres detalles que hacen que esto funcione bien: la velocidad se normaliza con la velocidad real de correr (los 7,5 m/s del jugador), se suaviza para que la mezcla no salte, y solo se llama a SetFloat cuando el valor cambió de verdad (con el hash del parámetro, no con el texto).

Decisión técnica: los clips animan el hijo Cuerpo y no la raíz del jugador. La raíz tiene el CharacterController, y animar su transform pelearía con la física del movimiento. Animando el hijo visual, la animación y la física conviven sin pisarse.

Las otras animaciones del juego (9 clips y 5 controladores, también generados por script) son las de los interactivos: portón que se hunde/sube, pop del mensaje de resultado, pulso de los círculos del marcador, entrada del panel de aviso y la inclinación del arquero al lanzarse.

10. Conclusión

El perfilado encontró basura por cuadro, no “scripts lentos”. Los dos problemas grandes eran de asignación de memoria: un array por cuadro en la cámara y un GameObject por sonido en el audio. Se corrigieron con un buffer reutilizable y un pool de voces.

La ganancia se mide, no se estima: 200 $\rightarrow$ 0 objetos por ráfaga de sonidos, 100 $\rightarrow$ 0 bytes por llamada de raycast, y el churn de la ventana de 6 s bajó de 203 a 2 objetos.

El tiempo de cuadro en el Editor es un mal indicador: está dominado por el render y el vsync (≈ 16 ms). Lo que importa en un juego en tiempo real es no generar basura, porque

las pausas del recolector se sienten como tirones; en una build de desarrollo eso se ve mejor en el módulo Memory del Profiler.

Hallazgos técnicos que quedaron documentados en el código (los encontramos resolviendo bugs reales del proyecto y sirven para no repetirlos):

- El Animator escribe el vector completo: si un clip anima solo la y de la posición, la x y la z se escriben como 0 (el portón terminó en el medio de la cancha). Se soluciona guardando las tres componentes, con las que no cambian fijas en su valor real.
- El bucle de un clip se controla con `AnimationClipSettings.loopTime`, no con `wrapMode`; si un clip de interactivo queda en bucle, se repite solo (el portón volvía a cerrar y dejaba al jugador encerrado en la cancha).
- Las animaciones de interfaz necesitan la curva guardada con el `classID` de `RectTransform` (224); con `SetCurve` quedan como `Transform` (4) y el Canvas las ignora en silencio.
- Con `Write Defaults` encendido, el Animator escribe 0 en todas las propiedades que el clip no anima. Conviene apagarlo en estados que animan solo una parte.